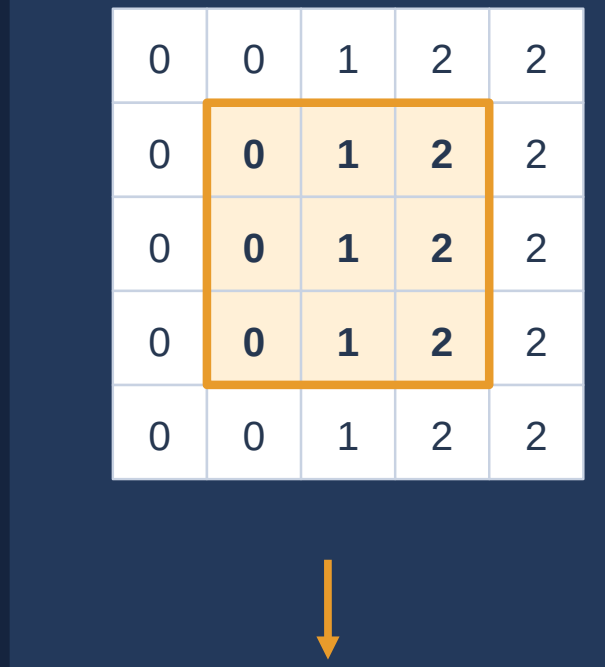
BEGINNER-FRIENDLY CNN LESSON

Convolution, explained one small step at a time

Understand filters, feature maps, stride, padding, channels, dilation, and PyTorch.

31 slides • 9 individual filter movements • 3 quick checks



A small filter scans the image

The whole idea in three pictures

A CNN looks for useful local patterns instead of reading each pixel independently.

1 · IMAGE

0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2

Pixel values

2 · SLIDING FILTER

-1	0	1
-1	0	1
-1	0	1

Same weights at every location

3 · FEATURE MAP

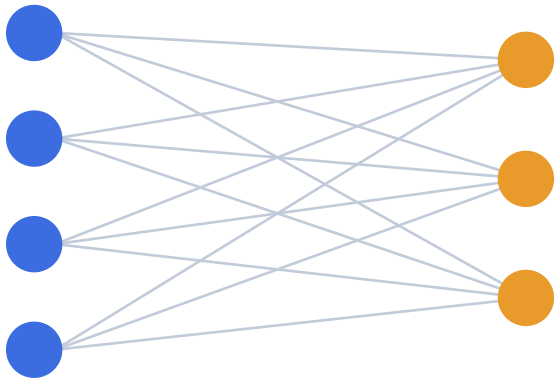
3	6	3
3	6	3
3	6	3

Where the pattern appears

Fully connected vs. convolutional layers

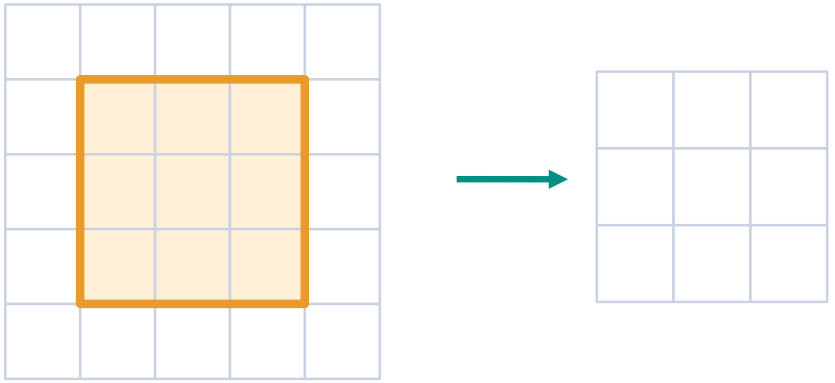
The key difference: a CNN uses small local connections and reuses a filter.

FULLY CONNECTED



Every input can connect to every output

CONVOLUTIONAL



Only a small neighborhood is read at a time

Remember: a CNN keeps the image's spatial layout while learning shared visual patterns.

Start with a tiny 5 × 5 image

Real images contain many more pixels, but small numbers let us see every calculation.

An image is simply a grid of numbers

0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2

5 rows × 5 columns

25 pixel values

Our example

Each row is 0, 0, 1, 2, 2

The values brighten from left to right. We will search for that change.

Meet the 3 × 3 filter (also called a kernel)

A filter is a small table of learnable weights. This one responds to left-to-right changes.

-1	0	1
-1	0	1
-1	0	1

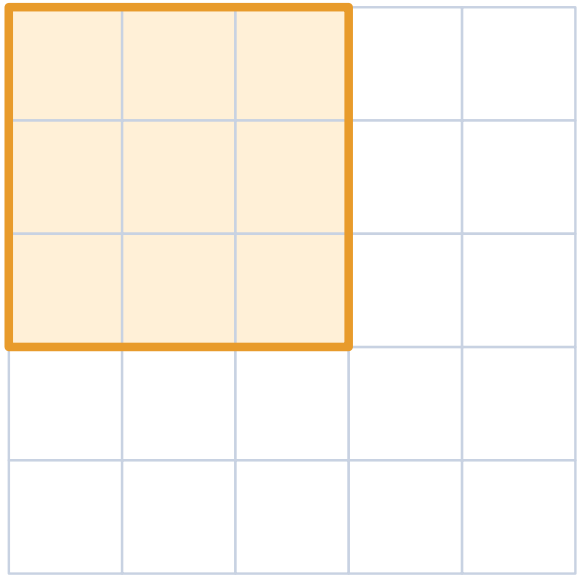
Each filter contains 9 weights for a single-channel image

−1 reads the left side • 0 ignores the middle • +1 reads the right side

At one position: multiply matching numbers, then add all 9 products.

The filter follows a simple sliding path

Stride = 1 means move exactly one input cell between nearby positions.



Output positions

1	2	3
4	5	6
7	8	9

1 → 2 → 3; then 4 → 5 → 6; then 7 → 8 → 9

Next: see each of those 9 positions, one slide at a time.

FILTER POSITION (1, 1)

Start at the top-left

One 3 × 3 filter covers exactly nine input numbers.

5 × 5 input image

0	**0**	**1**	2	2
0	**0**	**1**	2	2
0	**0**	**1**	2	2
0	0	1	2	2
0	0	1	2	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3		

Output (1, 1) = 3

One row: (−1×0) + (0×0) + (1×1) = 1 · Three identical rows → 3

FILTER POSITION (1, 2)

Move right by ONE cell

Every new position produces exactly ONE output number.

5 × 5 input image

0	**0**	**1**	**2**	2
0	**0**	**1**	**2**	2
0	**0**	**1**	**2**	2
0	0	1	2	2
0	0	1	2	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	**6**	

Output (1, 2) = 6

One row: (−1×0) + (0×1) + (1×2) = 2 · Three identical rows → 6

FILTER POSITION (1, 3)

Move right one more cell

Every new position produces exactly ONE output number.

5 × 5 input image

0	0	**1**	**2**	**2**
0	0	**1**	**2**	**2**
0	0	**1**	**2**	**2**
0	0	1	2	2
0	0	1	2	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	6	**3**

Output (1, 3) = 3

One row: (−1×1) + (0×2) + (1×2) = 1 · Three identical rows → 3

FILTER POSITION (2, 1)

Move down to the next row

When we finish a row, the filter returns to the left.

5 × 5 input image

0	0	1	2	2
0	**0**	**1**	2	2
0	**0**	**1**	2	2
0	**0**	**1**	2	2
0	0	1	2	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	6	3
3		

Output (2, 1) = 3

One row: (−1×0) + (0×0) + (1×1) = 1 · Three identical rows → 3

FILTER POSITION (2, 2)

Move to the middle

Every new position produces exactly ONE output number.

5 × 5 input image

0	0	1	2	2
0	**0**	**1**	**2**	2
0	**0**	**1**	**2**	2
0	**0**	**1**	**2**	2
0	0	1	2	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	6	3
3	**6**	

Output (2, 2) = 6

One row: (−1×0) + (0×1) + (1×2) = 2 · Three identical rows → 6

FILTER POSITION (2, 3)

Finish the second row

Every new position produces exactly ONE output number.

5 × 5 input image

0	0	1	2	2
0	0	**1**	**2**	**2**
0	0	**1**	**2**	**2**
0	0	**1**	**2**	**2**
0	0	1	2	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	6	3
3	6	**3**

Output (2, 3) = 3

One row: (−1×1) + (0×2) + (1×2) = 1 · Three identical rows → 3

FILTER POSITION (3, 1)

Begin the final row

When we finish a row, the filter returns to the left.

5 × 5 input image

0	0	1	2	2
0	0	1	2	2
0	**0**	**1**	2	2
0	**0**	**1**	2	2
0	**0**	**1**	2	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	6	3
3	6	3
3		

Output (3, 1) = 3

One row: (−1×0) + (0×0) + (1×1) = 1 · Three identical rows → 3

FILTER POSITION (3, 2)

Move right again

Every new position produces exactly ONE output number.

5 × 5 input image

0	0	1	2	2
0	0	1	2	2
0	**0**	**1**	**2**	2
0	**0**	**1**	**2**	2
0	**0**	**1**	**2**	2

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	6	3
3	6	3
3	**6**	

Output (3, 2) = 6

One row: (−1×0) + (0×1) + (1×2) = 2 · Three identical rows → 6

FILTER POSITION (3, 3)

Last position: bottom-right

Every new position produces exactly ONE output number.

5 × 5 input image

0	0	1	2	2
0	0	1	2	2
0	0	**1**	**2**	**2**
0	0	**1**	**2**	**2**
0	0	**1**	**2**	**2**

3 × 3 filter

-1	0	1
-1	0	1
-1	0	1

multiply + add

3 × 3 output

3	6	3
3	6	3
3	6	**3**

Output (3, 3) = 3

One row: (−1×1) + (0×2) + (1×2) = 1 · Three identical rows → 3

Nine positions → nine output values

The 5 × 5 input becomes a 3 × 3 feature map.

Input

0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2

Filter

-1	0	1
-1	0	1
-1	0	1

Completed output

3	6	3
3	6	3
3	6	3

Each output cell comes from a different 3 × 3 input patch — same filter, new location.

Predict the output size before calculating

For an ordinary convolution with dilation = 1, use this rule for height and width.

$$\text{Output size} = \text{floor}[(n + 2p - f) / s] + 1$$

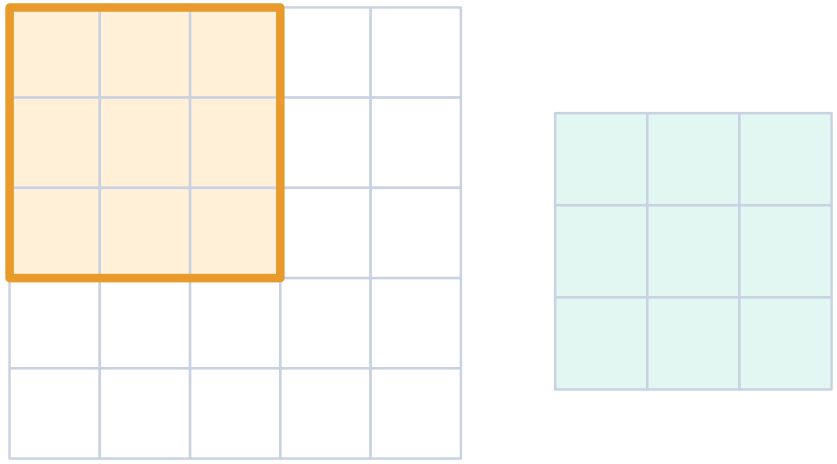
n	input size = 5
f	filter size = 3
p	padding = 0
s	stride = 1

$$\text{floor}[(5 + 0 - 3) / 1] + 1 = 3 \rightarrow 3 \times 3 \text{ output}$$

Stride controls how far we jump

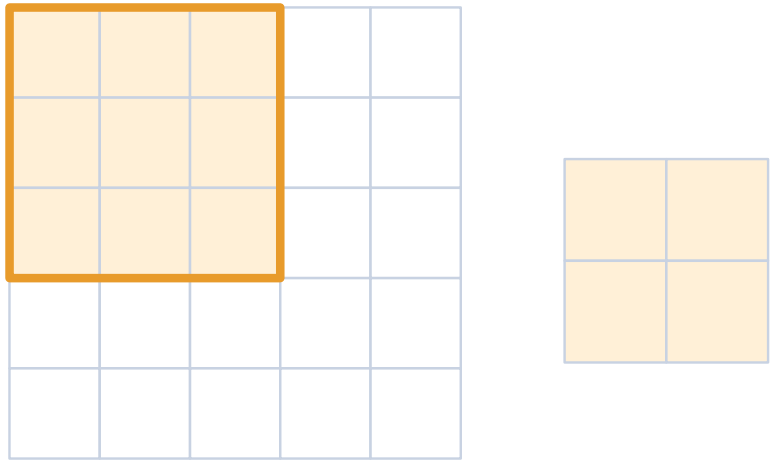
Same input. Same filter. Different stride → a different number of output positions.

STRIDE = 1



3 × 3 output · 9 positions

STRIDE = 2



2 × 2 output · 4 positions

Bigger stride = fewer positions and typically a smaller feature map.

Padding: add a border so the image stays larger

Padding = 1 adds one ring of zeros around all four sides.

Original: 5 × 5

0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2
0	0	1	2	2

After zero padding: 7 × 7

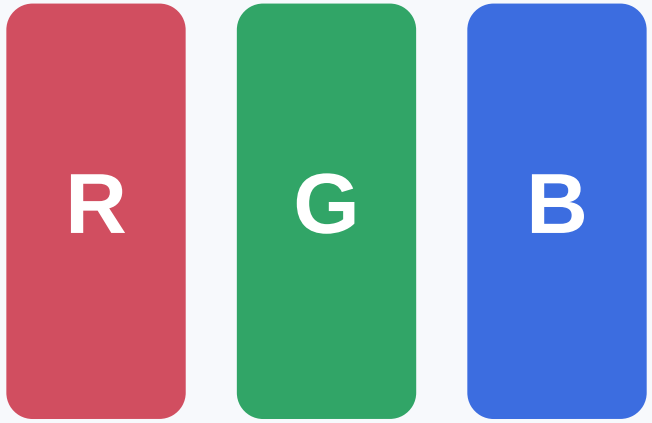
0	0	0	0	0	0	0
0	0	0	1	2	2	0
0	0	0	1	2	2	0
0	0	0	1	2	2	0
0	0	0	1	2	2	0
0	0	0	1	2	2	0
0	0	0	0	0	0	0

3 × 3 filter
stride = 1
padding = 1

Output = (5 + 2×1 – 3) / 1 + 1 = 5

What changes when the image has color?

An RGB image has three channels: red, green, and blue.



Input image

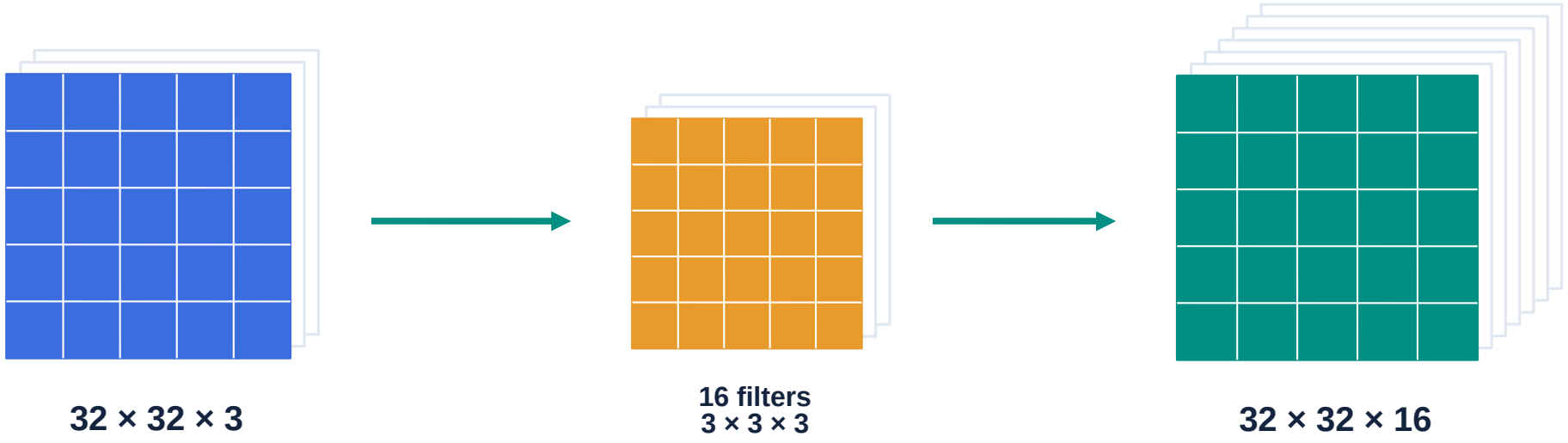
One filter must cover ALL 3 channels

3 × 3 × 3 = 27 learned weights

One complete 3D filter → one 2D feature map.

First CNN layer: 16 filters make 16 maps

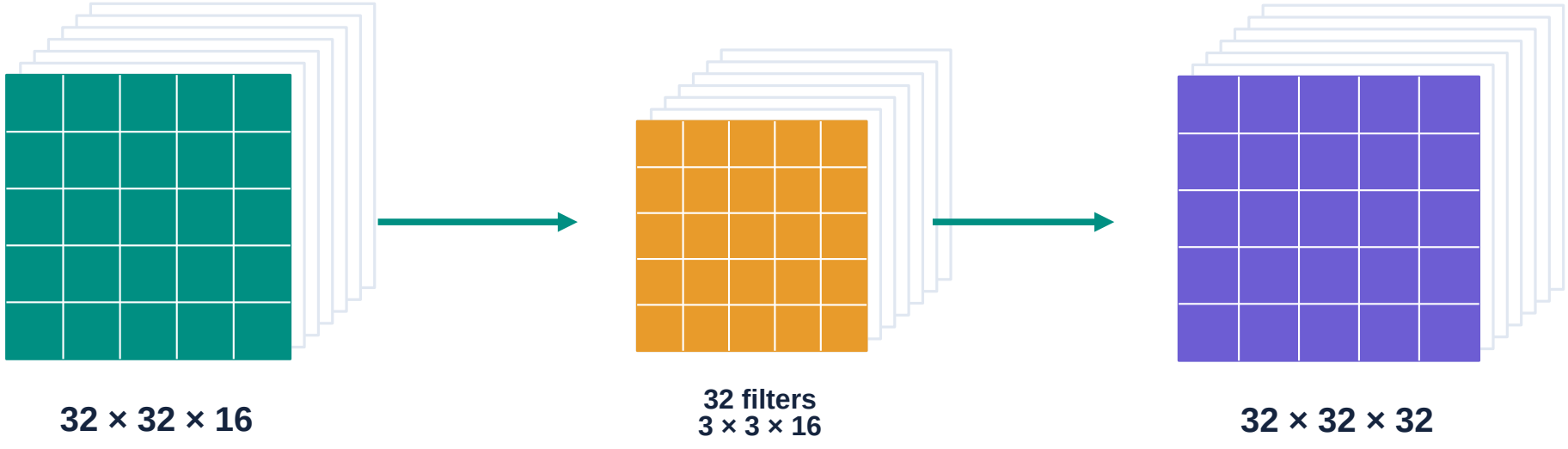
Example from the notes: preserve width and height using a 3 × 3 filter with padding = 1.



16 separate filters = 16 different feature maps

Second CNN layer: the filter gets deeper

The NEXT filter spans all 16 feature maps coming from the first layer.



Filter depth = input channels. Number of filters = output channels.

Why use many filters?

Different filters learn different patterns from the same input.

Filter A

-1	0	1
-1	0	1
-1	0	1

Vertical changes

Filter B

-1	-1	-1
0	0	0
1	1	1

Horizontal changes

Filter C

0	1	0
1	-4	1
0	1	0

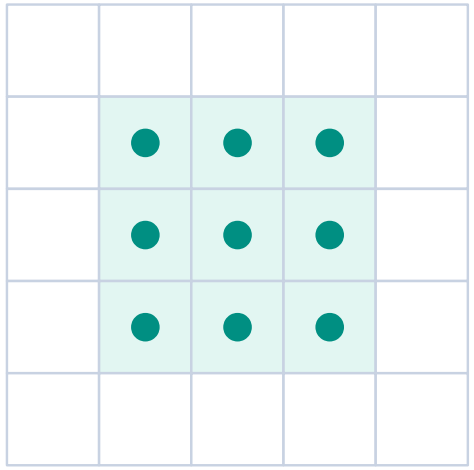
Other textures

These are illustrative filters. During training, CNNs learn their own weights.

Dilation: see farther without a bigger filter

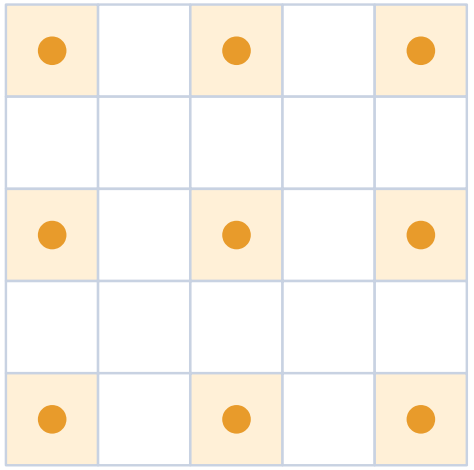
Dilation = 2 spaces the 3 × 3 weights apart across a 5 × 5 region.

DILATION = 1



9 weights across a 3 × 3 region

DILATION = 2



9 weights across a 5 × 5 region

Effective filter width = dilation × (kernel size − 1) + 1

Now write the same idea in PyTorch

This is a complete two-layer convolution example from the concepts in your notes.

```
import torch
from torch import nn

model = nn.Sequential(
  nn.Conv2d(3,16,kernel_size=3,padding=1),
  nn.ReLU(),
  nn.Conv2d(16,32,kernel_size=3,padding=1),
  nn.ReLU()
)
```

First layer

3 input channels → 16 outputs

Second layer

16 input channels → 32 outputs

nn.Conv2d(in_channels, out_channels, kernel_size, ...)

PyTorch shape order: N × C × H × W

This is NOT the same order as the classroom diagrams (H × W × C).

N	C	H	W
batch size	channels	height	width

Input: (1, 3, 32, 32)

Output: (1, 32, 32, 32)

One image in → 32 learned feature maps out, all 32 × 32.

How does a CNN finally make a decision?

After extracting features, a classifier can turn feature values into class scores.



Convolution extracts features; a fully connected layer can use them to classify.

Avoid these four common mix-ups

Checking the vocabulary prevents most CNN shape mistakes.

Filter size

3 × 3 is the spatial window, not the output image size.

Filter depth

The filter must span every input channel.

Number of filters

This determines the number of OUTPUT channels.

Stride and padding

These mainly control height and width of the output.

Quick class check (30 seconds each)

1. **5 × 5 input, 3 × 3 filter, stride 1, padding 0**

   Output shape: 3 × 3 or 5 × 5?

2. **32 × 32 × 3 input, 16 filters, padding 1**

   How many output channels?

3. **Second layer gets 16 input channels**

   What depth must each filter have?

Answer key: explain WHY, not only WHAT

All three answers follow the same simple rules.

1	**3 × 3 output**	(5 − 3) / 1 + 1 = 3 for each side.
2	**16 output channels**	One output map per filter → 16 maps.
3	**Depth = 16**	Each second-layer filter spans all 16 input maps.

Keep this CNN mental model

A filter slides, produces a map, and deeper layers learn more feature maps.

SLIDE

Move filter across the image

MULTIPLY + ADD

One number per filter position

STACK

One output map per filter

REPEAT

The next layer sees all previous maps

32 × 32 × 3 → 32 × 32 × 16 → 32 × 32 × 32